

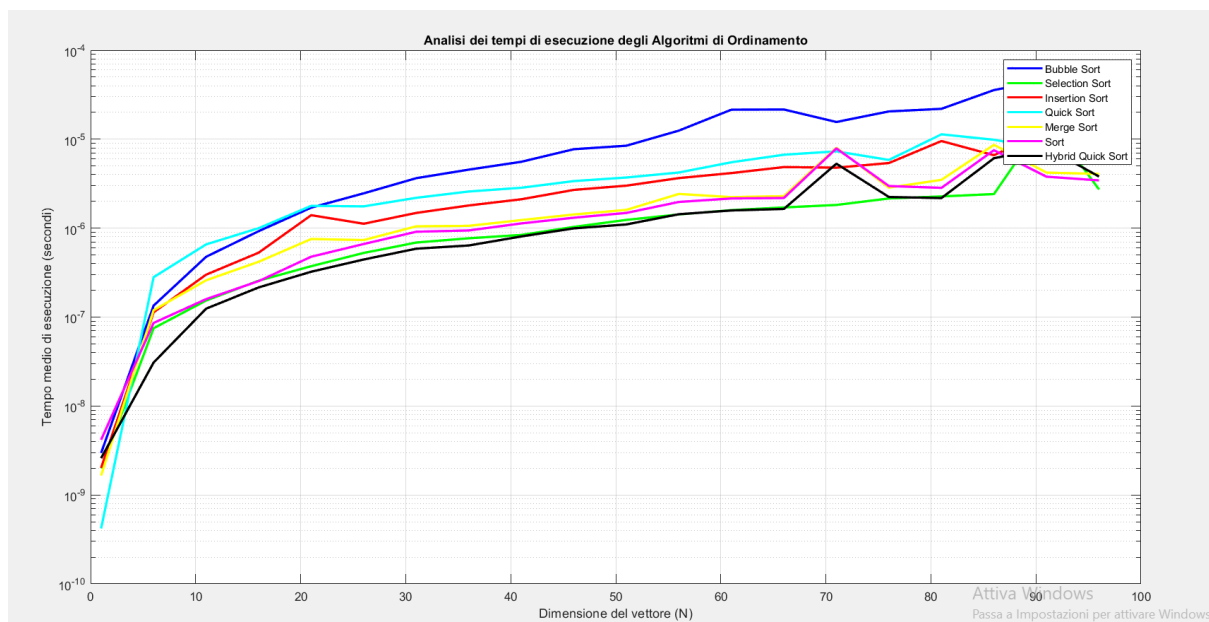
In questa esercitazione sono stati implementati e confrontati diversi algoritmi di ordinamento. Per ciascuno di essi sono stati eseguiti test unitari su 100 vettori al fine di verificarne la correttezza; successivamente sono stati misurati i tempi di esecuzioni al variare della dimensione N dei vettori.

A seguito dell'integrazione del programma *measure_sort*, è stata condotta un'analisi accurata delle prestazioni degli algoritmi di ordinamento al variare delle dimensioni N del vettore di input. Le misurazioni sono state effettuate su 10 vettori di dimensioni inferiore a 100.

Nel file *measure_sort* è stata inizializzata una struttura dati di tipo **vector<vector<int>>**, ovvero una matrice composta da 100 righe e 10 colonne, successivamente riempita con valori casuali tramite il comando **rf.fill**.

Per ciascun test è stato quindi implementato un ciclo *for* per misurare il tempo impiegato dall'algoritmo nell'ordinamento dei vettori generati; al termine, è stata calcolata la media dei tempi registrati per ogni dimensione considerata.

I risultati ottenuti sono stati visualizzati tramite un grafico realizzato in MATLAB, con l'asse delle ordinate in scala logaritmica.



Dall'osservazione del grafico, è possibile condurre un'analisi delle prestazioni degli algoritmi al variare delle dimensioni N del vettore.

Si distinguono tre fasi principali nel comportamento degli algoritmi:

- Per $N \in [1;10]$ le differenze tra gli algoritmi sono minime e le curve risultano molto ravvicinate. In questo intervallo, l'**Insertion Sort** e l'**Hybrid Quick Sort** mostrano i tempi più bassi, mentre il **Bubble Sort** risulta già il più lento.
- Per $N \in [11;45]$ il **Bubble Sort** si consolida come l'algoritmo meno efficiente. L'**Insertion Sort** mantiene una crescita moderata, risultando il migliore tra gli

algoritmi quadratici. Il **Quick Sort** e il **Merge Sort** mostrano già una pendenza inferiore in scala logaritmica.

- Per valori di $N \geq 45$ gli algoritmi logaritmici (Merge, Quick, Sort) evidenziano una crescita nettamente più lenta rispetto a quella degli algoritmi quadratici. Il **Bubble Sort** è quello che cresce più rapidamente, diventando di gran lunga il più lento all'aumentare di N .

Dalle misurazioni effettuate, la soglia n_0 a partire dalla quale gli algoritmi logaritmici risultano più convenienti si colloca intorno a 40-45.

Come si evince dal grafico, la curva dell'**Hybrid Quicksort** per valori piccoli di N , presenta prestazioni molto simili a quelle dell'**Insertion Sort**, sfruttandone l'efficienza su array di piccole dimensioni.

Al crescere di N , invece, segue invece l'andamento logaritmico tipico del **Quick Sort**, mantenendosi generalmente al di sotto della sua curva e risultando quindi complessivamente più efficiente rispetto alla versione tradizionale.

L'esercitazione ha permesso di verificare concretamente come le differenze teoriche tra gli algoritmi si riflettano nei tempi di esecuzioni osservati.

Per dimensioni di vettore ridotte, algoritmi semplici come l'**Insertion Sort** risultano più efficienti degli algoritmi logaritmici, poiché quest' ultimi introducono costi aggiuntivi legati alla ricorsione e alla gestione della memoria.

Al crescere di N , tuttavia, il vantaggio asintotico degli algoritmi logaritmici diventa evidente. L'**Hybrid Quick Sort** rappresenta una soluzione efficace in quanto sfrutta entrambi i comportamenti: usa una strategia semplice sui vettori piccoli e una più efficiente su quelli grandi, ottenendo prestazioni complessivamente migliori.